code/nndl/cnn.py

```python
import numpy as np

from cs231n.fast_layers import (
    conv_forward_fast,
    conv_backward_fast,
    max_pool_forward_fast,
    max_pool_backward_fast,
)

from nndl.layers import (
    affine_forward,
    affine_backward,
    relu_forward,
    relu_backward,
    softmax_loss,
    batchnorm_forward,
    batchnorm_backward,
)
from nndl.conv_layers import (
    conv_forward_naive,
    conv_backward_naive,
    max_pool_forward_naive,
    max_pool_backward_naive,
    spatial_batchnorm_forward,
    spatial_batchnorm_backward,
)
from nndl.layer_utils import (
    affine_relu_forward,
    affine_relu_backward,
)
from nndl.conv_layer_utils import (
    conv_relu_forward,
    conv_relu_backward,
    conv_relu_pool_forward,
    conv_relu_pool_backward,
)


"""
This code was originally written for CS 231n at Stanford University
(cs231n.stanford.edu).  It has been modified in various areas for use in the
ECE 239AS class at UCLA.  This includes the descriptions of what code to
implement as well as some slight potential changes in variable names to be
consistent with class nomenclature.  We thank Justin Johnson & Serena Yeung for
permission to use this code.  To see the original version, please visit
cs231n.stanford.edu.
"""


class ThreeLayerConvNet:
    """
```

```python
    A three-layer convolutional network with the following architecture:

    conv - relu - 2x2 max pool - affine - relu - affine - softmax

    The network operates on minibatches of data that have shape (N, C, H, W)
    consisting of N images, each with height H and width W and with C input
    channels.
    """

    def __init__(
        self,
        input_dim: tuple[int, int, int] = (3, 32, 32),
        num_filters: int = 32,
        filter_size: int = 7,
        hidden_dim: int = 100,
        num_classes: int = 10,
        weight_scale: float = 1e-3,
        reg: float = 0.0,
        dtype: np.dtype = np.float32,
        use_batchnorm: bool = False,
    ):
        """
        Initialize a new network.

        Inputs:
        - input_dim: Tuple (C, H, W) giving size of input data
        - num_filters: Number of filters to use in the convolutional layer
        - filter_size: Size of filters to use in the convolutional layer
        - hidden_dim: Number of units to use in the fully-connected hidden layer
        - num_classes: Number of scores to produce from the final affine layer.
        - weight_scale: Scalar giving standard deviation for random initialization
          of weights.
        - reg: Scalar giving L2 regularization strength
        - dtype: numpy datatype to use for computation.
        """
        self.use_batchnorm = use_batchnorm
        self.params = {}
        self.reg = reg
        self.dtype = dtype

        # ================================================================ #
        # YOUR CODE HERE:
        #   Initialize the weights and biases of a three layer CNN. To initialize:
        #     - the biases should be initialized to zeros.
        #     - the weights should be initialized to a matrix with entries
        #         drawn from a Gaussian distribution with zero mean and
        #         standard deviation given by weight_scale.
        # ================================================================ #

        C, H, W = input_dim

        self.params['W1'] = weight_scale * np.random.randn(num_filters, C, filter_size, filter_size)
        self.params['b1'] = np.zeros(num_filters)
```

```python

        pool_out_height = H // 2
        pool_out_width = W // 2
        fc_input_dim = num_filters * pool_out_height * pool_out_width

        self.params['W2'] = weight_scale * np.random.randn(fc_input_dim, hidden_dim)
        self.params['b2'] = np.zeros(hidden_dim)

        self.params['W3'] = weight_scale * np.random.randn(hidden_dim, num_classes)
        self.params['b3'] = np.zeros(num_classes)

        if self.use_batchnorm:
            self.params['gamma1'] = np.ones(num_filters)
            self.params['beta1'] = np.zeros(num_filters)
            self.params['gamma2'] = np.ones(hidden_dim)
            self.params['beta2'] = np.zeros(hidden_dim)

        # ================================================================ #
        # END YOUR CODE HERE
        # ================================================================ #

        for k, v in self.params.items():
            self.params[k] = v.astype(dtype)

    def loss(self, X, y=None):
        """
        Evaluate loss and gradient for the three-layer convolutional network.

        Input / output: Same API as TwoLayerNet in fc_net.py.
        """
        W1, b1 = self.params["W1"], self.params["b1"]
        W2, b2 = self.params["W2"], self.params["b2"]
        W3, b3 = self.params["W3"], self.params["b3"]

        # pass conv_param to the forward pass for the convolutional layer
        filter_size = W1.shape[2]
        conv_param = {"stride": 1, "pad": (filter_size - 1) // 2}

        # pass pool_param to the forward pass for the max-pooling layer
        pool_param = {"pool_height": 2, "pool_width": 2, "stride": 2}

        scores = None

        # ================================================================ #
        # YOUR CODE HERE:
        #   Implement the forward pass of the three layer CNN.  Store the output
        #   scores as the variable "scores".
        # ================================================================ #

        out1, cache1 = conv_relu_pool_forward(X, W1, b1, conv_param, pool_param)
        out2, cache2 = affine_relu_forward(out1, W2, b2)
        scores, cache3 = affine_forward(out2, W3, b3)
```

```python

        # ================================================================ #
        # END YOUR CODE HERE
        # ================================================================ #

        if y is None:
            return scores

        loss, grads = 0, {}
        # ================================================================ #
        # YOUR CODE HERE:
        #   Implement the backward pass of the three layer CNN.  Store the grads
        #   in the grads dictionary, exactly as before (i.e., the gradient of
        #   self.params[k] will be grads[k]).  Store the loss as "loss", and
        #   don't forget to add regularization on ALL weight matrices.
        # ================================================================ #

        loss, dscores = softmax_loss(scores, y)
        loss += 0.5 * self.reg * (np.sum(W1**2) + np.sum(W2**2) + np.sum(W3**2))

        dout2, dW3, db3 = affine_backward(dscores, cache3)
        dout1, dW2, db2 = affine_relu_backward(dout2, cache2)
        dx, dW1, db1 = conv_relu_pool_backward(dout1, cache1)

        grads['W3'] = dW3 + self.reg * W3
        grads['b3'] = db3
        grads['W2'] = dW2 + self.reg * W2
        grads['b2'] = db2
        grads['W1'] = dW1 + self.reg * W1
        grads['b1'] = db1

        # ================================================================ #
        # END YOUR CODE HERE
        # ================================================================ #

        return loss, grads

```